

Making It Fast: the KV Cache

Why generation doesn't start over for every token · video 2:07

This bonus lesson is about speed. Generating text means one model run per new token, and done naively every run **redoes the work for the whole text**. The **KV cache** saves each token's keys and values, so every step only has to process **one new token**. The price is memory.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 10).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. The naive loop repeats work	▶ 0:08-0:21	4	☐
2. Old tokens never change	▶ 0:21-0:34	4	☐
3. The KV cache: one new row per step	▶ 0:34-1:04	4	☐
4. The price: memory	▶ 1:04-1:20	4	☐
5. Prefill and decode	▶ 1:20-1:30	4	☐
6. Shrinking the cache: grouped-query attention	▶ 1:30-1:41	4	☐
7. The KV cache in code	▶ 1:41-1:54	3	☐

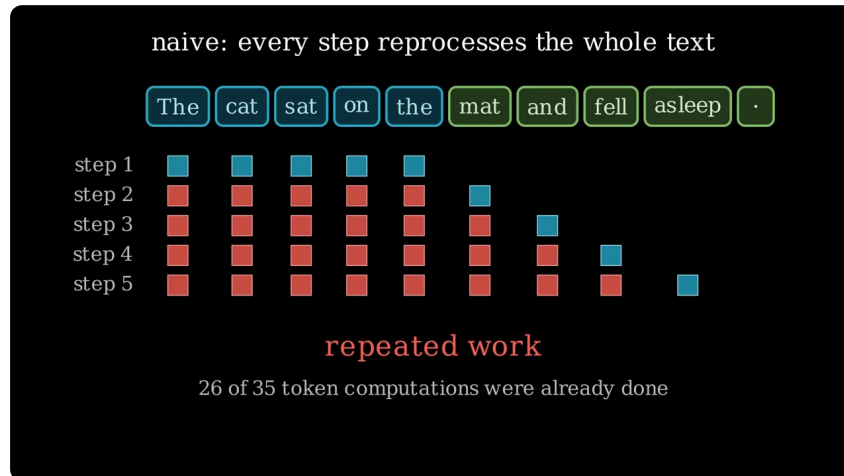
Before you start: this lesson builds on episode 1 (the generation loop), episode 6 (queries, keys, values and the causal mask), episode 7 (attention heads) and episode 9 (layers stacked in blocks). Concept 7 reads NumPy code; Foundations F11 (NumPy in Three Minutes) helps.

CONCEPT 1 OF 7

The naive loop repeats work

▶ Video 0:08-0:21

Generating text means running the model **once for every new token** (the loop of episode 1). Done naively, each run processes the **whole text again**, from the very first token. With the 5-token prompt “The cat sat on the”, five steps process $5 + 6 + 7 + 8 + 9 = 35$ tokens, and 26 of those computations had already been done before: **repeated work**.



▶ 0:20 Naive generation: each step processes the whole text again. The red squares were already computed on an earlier step.

KEY IDEA

Without a cache, every new token costs a run over the **entire text so far**. The work per step keeps growing, and most of it repeats earlier steps.

Check 1: The naive loop repeats work

Answer all 4 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

26 of the video's 35 naive computations are repeats. How many are new, and where do they come from?

1.2 CALCULATION

A new case: a 10-token prompt and 4 naive steps. How many token computations in total?

1.3 TRUE OR FALSE

“In the naive loop, every step costs the same, however long the text is.”

True ☐ False ☐

1.4 MULTIPLE CHOICE

What exactly makes the naive loop wasteful?

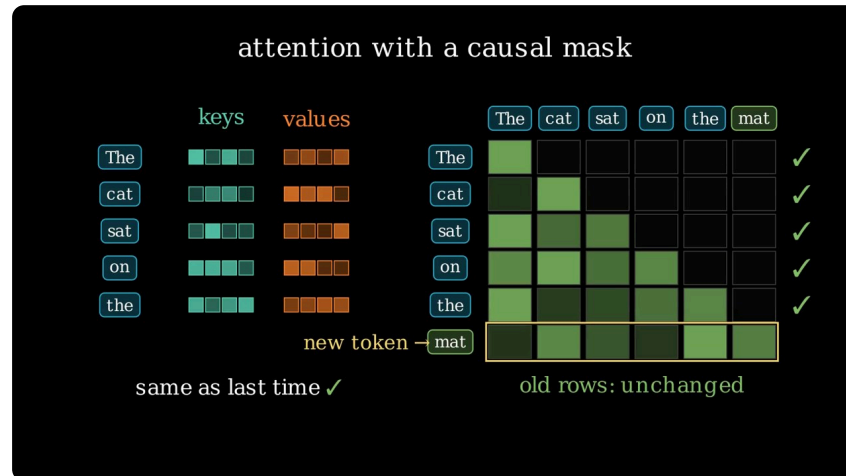
- A. It predicts several tokens at once
- B. It redoes the old tokens' work, already done on earlier steps
- C. It forgets the prompt after the first step
- D. It uses the causal mask

CONCEPT 2 OF 7

Old tokens never change

▶ Video 0:21-0:34

With the **causal mask** (episode 6), every token looks only at itself and the tokens **before** it, never at later ones. So when a new token arrives, nothing about the old tokens changes: the grid gains **one new row** and a masked column, and the old tokens' **keys and values are exactly the same as last time**, in every layer.



▶ 0:33 A new token (mat) adds one row and one masked column. The old rows are unchanged, and so are the old keys and values.

KEY IDEA

A token's keys and values depend only on that token and the ones before it. Adding a token **never changes** anything about the earlier ones.

Check 2: Old tokens never change

Answer all 4 questions before looking at the answers. Pass mark: all correct.

2.1 MULTIPLE CHOICE

A sixth token arrives in a 5×5 causal attention grid. What happens to the grid?

- A. Every row is recomputed with new weights
- B. A new row and a masked column are added; the old rows stay the same
- C. Only the first row changes
- D. The oldest token is dropped

2.2 CALCULATION

In that same step (5×5 grid $\rightarrow 6 \times 6$), how many cells are new, and how many of the new cells hold real (non-zero) attention weights?

2.3 TRUE OR FALSE

"When *mat* is added, the key of *cat* must be recomputed, because *cat* can now attend to *mat*."

True ☐ False ☐

2.4 SHORT ANSWER

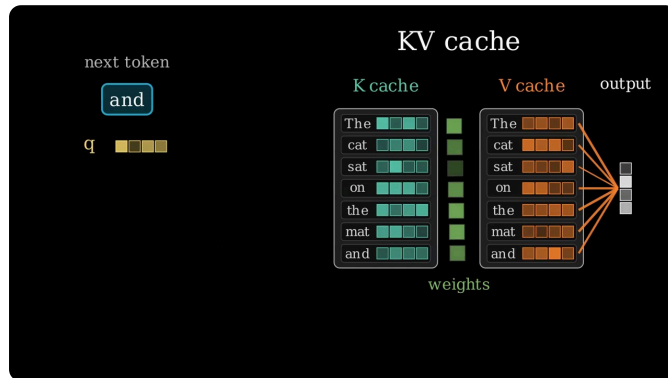
Imagine a model **without** the causal mask. When a new token arrives, would the old tokens' keys and values in the later layers stay the same? Explain.

CONCEPT 3 OF 7

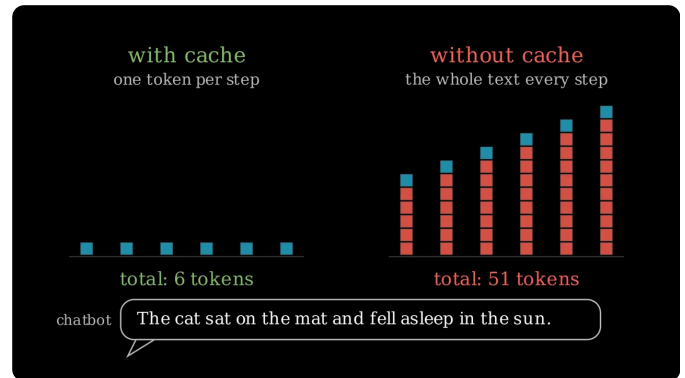
The KV cache: one new row per step

▶ Video 0:34–1:04

So we save them: the **KV cache** stores every token's **key** and **value**, in every layer. When the next token arrives, we compute only **its own** query, key and value. Its query is compared with **all the cached keys**, and the weights mix **the cached values**: one new row of the grid, not the whole grid. Each step now processes **one token** instead of the entire text, a big part of why chatbots can stream their answers so quickly.



▶ 0:52 The new token's query is compared with all cached keys; the weights mix the cached values into its output.



▶ 1:03 Six steps: one token per step with the cache (6 in total), the whole text every step without it (51).

KEY IDEA

Each new token computes only its own query, key and value, then attends over the cached keys and values: **one new row per step**, with exactly the same result as recomputing everything.

Check 3: The KV cache: one new row per step

Answer all 4 questions before looking at the answers. Pass mark: all correct.

3.1 MULTIPLE CHOICE

With a KV cache, which vectors are computed when the next token arrives?

- A. A query, key and value for every token
- B. Only the new token's query, key and value
- C. Only the new token's key and value; its query is cached
- D. All keys and values, no queries

3.2 SHORT ANSWER

The cache stores keys and values, but no queries. Why is an old token's query never needed again?

3.3 CALCULATION

A new case: 100 steps from a 1-token prompt, so step k sees k tokens. How many token computations (a) without a cache, (b) with one?

3.4 TRUE OR FALSE

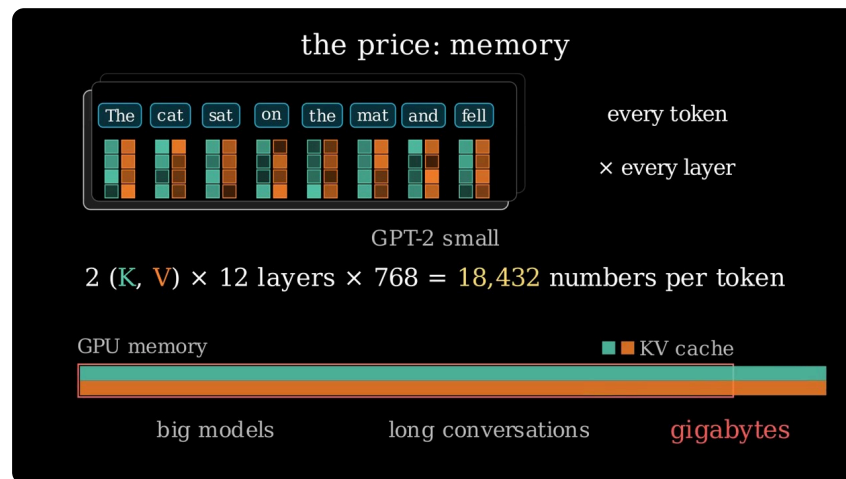
"The KV cache gives a slightly different, approximate result."

True ☐ False ☐

The price: memory

▶ Video 1:04-1:20

Speed is paid for with **memory**. The cache holds a key and a value **for every token, in every layer**. GPT-2 small has 12 layers and vectors of 768 numbers, so each token adds **2 (K and V) × 12 × 768 = 18,432 numbers**. For big models and long conversations, the cache can take up **gigabytes**.



▶ 1:19 Every token, in every layer: 18,432 numbers per token for GPT-2 small, and gigabytes for big models and long chats.

KEY IDEA

Cache size = **2 × layers × vector size** numbers per token, times the number of tokens. It grows with every token of the conversation.

Check 4: The price: memory

Answer all 4 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

How many numbers does GPT-2 small's cache hold for a conversation of 1,000 tokens?

4.2 CALCULATION

A new case: 32 layers and vectors of 4,096 numbers. (a) How many numbers per token? (b) At 2 bytes per number, how much memory for 8,000 tokens?

4.3 MULTIPLE CHOICE

The conversation becomes twice as long. What happens to the size of the cache?

- A. It stays the same
- B. It doubles
- C. It quadruples
- D. It halves

4.4 TRUE OR FALSE

"The cache stores one key and one value per token, shared by all the layers."

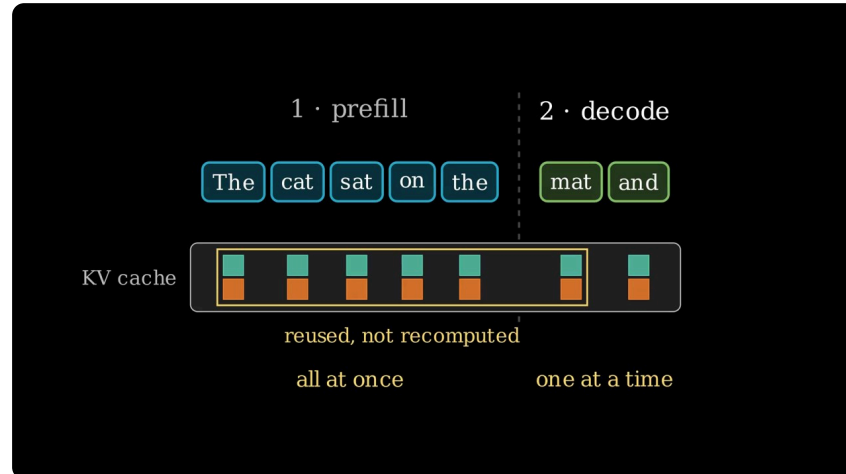
True ☐ False ☐

CONCEPT 5 OF 7

Prefill and decode

▶ Video 1:20-1:30

So generation has two phases. **Prefill**: read the **whole prompt at once** and fill the cache with its keys and values. The prompt is known in advance, so all its tokens can be processed together, in parallel (as in episode 6). Then **decode**: **one token at a time**, reusing the cache.



▶ 1:29 Prefill reads the whole prompt at once and fills the cache; decode adds one token at a time, reusing what is cached.

KEY IDEA

Prefill: the whole prompt in one go, filling the cache. **Decode**: one token per step, reusing the cache.

Check 5: Prefill and decode

Answer all 4 questions before looking at the answers. Pass mark: all correct.

5.1 PUT IN ORDER

Put in order: *decode, one token per step · the user sends a prompt · the cache now holds the prompt's keys and values · prefill.*

5.2 TRUE OR FALSE

"During prefill, the prompt's tokens are fed through the model one at a time."

True ☐ False ☐

5.3 SHORT ANSWER

Why can prefill handle all the prompt's tokens together, while decode must go one token at a time?

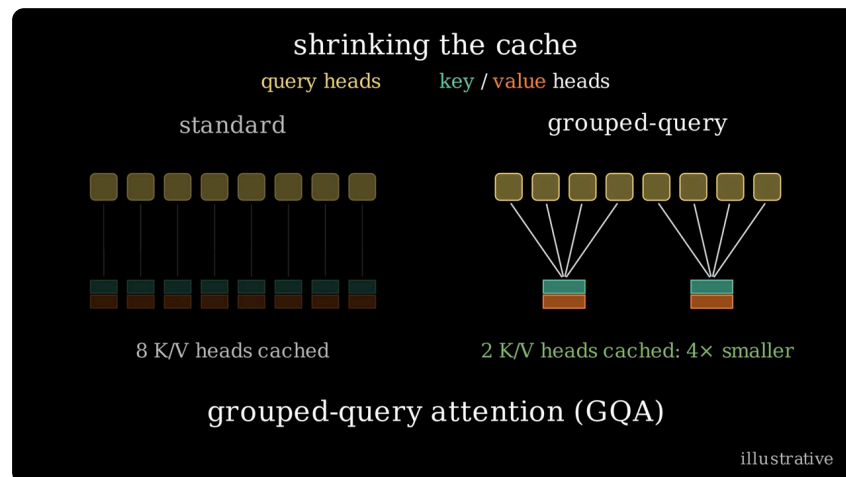
5.4 CALCULATION

A new case: GPT-2 small receives a 200-token prompt. (a) How many tokens does each decode step process? (b) How many numbers are in the cache right after prefill?

Shrinking the cache: grouped-query attention

▶ Video 1:30-1:41

Because the cache gets so large, modern models **shrink** it. Attention has several **heads** (episode 7), each with its own queries, keys and values. **Grouped-query attention** (GQA) lets **several query heads share the same keys and values**. In the video's illustration, 8 query heads share 2 key/value heads, so the cache is **4× smaller**. Every head still asks its own question.



▶ 1:40 Standard: every query head has its own keys and values. Grouped-query: groups of query heads share them (illustrative).

KEY IDEA

GQA: groups of query heads share one key/value head. The cache shrinks by the factor **query heads ÷ key/value heads**.

Check 6: Shrinking the cache: grouped-query attention

Answer all 4 questions before looking at the answers. Pass mark: all correct.

6.1 CALCULATION

A model has 32 query heads and 8 key/value heads. How many query heads share each key/value head, and how much smaller is the cache than with standard attention?

6.2 CALCULATION

GPT-2 small has 12 layers, each with 12 heads of 64 numbers ($12 \times 64 = 768$). If it used GQA with 4 key/value heads, how many numbers would its cache hold per token?

6.3 TRUE OR FALSE

“Grouped-query attention works by using fewer query heads.”

True ☐ False ☐

6.4 MULTIPLE CHOICE

Why do modern models use grouped-query attention?

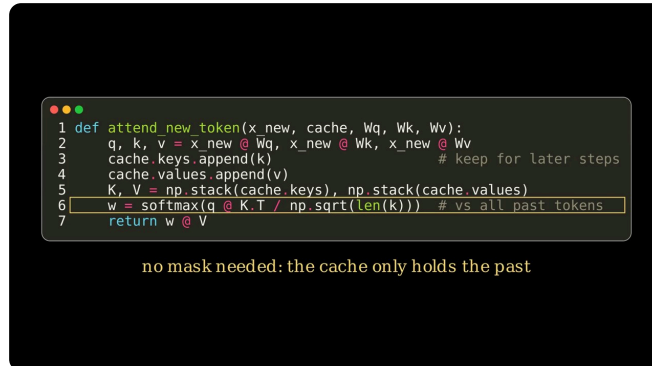
- A. To make each head more accurate
- B. To make the KV cache smaller
- C. So that no causal mask is needed
- D. To make the prompt shorter

CONCEPT 7 OF 7

The KV cache in code

▶ Video 1:41-1:54

The whole idea fits in one small function. `x_new` is the new token's vector and `cache` holds two lists, `keys` and `values`.



▶ 1:53 `attend_new_token` from the video: one new row of attention, using the cache.

```
def attend_new_token(x_new, cache, Wq, Wk, Wv):
    q, k, v = x_new @ Wq, x_new @ Wk, x_new @ Wv
    cache.keys.append(k)          # keep for later steps
    cache.values.append(v)
    K, V = np.stack(cache.keys), np.stack(cache.values)
    w = softmax(q @ K.T / np.sqrt(len(k))) # vs all past tokens
    return w @ V
```

Compute the new token's query, key and value; append the key and value to the cache; then attend over everything cached so far. **No mask is needed**, because the cache only holds the past (and the new token itself).

KEY IDEA

One call = one new row of the attention grid: the scaled score of `q` against every cached key, a softmax, and a weighted mix of the cached values.

Check 7: The KV cache in code

Answer all 3 questions before looking at the answers. Pass mark: all correct.

7.1 SHORT ANSWER

Keys have 64 numbers each. After `attend_new_token` has run for the 7 tokens *The cat sat on the mat and*, what are the shapes of `K` and of `w`?

7.2 MULTIPLE CHOICE

Why does `attend_new_token` need no causal mask?

- A. Masks are only needed during training
- B. The cache only holds the new token and the ones before it, so there is no future to hide
- C. The softmax ignores future tokens by itself
- D. Dividing by `np.sqrt(len(k))` removes them

7.3 HANDS-ON CODE

Try it yourself. The code below puts episode 6's full causal `attention` next to this episode's `attend_new_token`, and feeds 5 random tokens through the cache one at a time. (a) What does the first `print` show, and what does it tell you? (b) What does the second `print` show? (c) Delete the line in `attention` that applies the mask and run it again. What changes, and why?

```
import numpy as np

def softmax(s):
    e = np.exp(s - s.max(-1, keepdims=True))
    return e / e.sum(-1, keepdims=True)

def attention(X, Wq, Wk, Wv):          # episode 6: the whole grid, masked
    Q, K, V = X @ Wq, X @ Wk, X @ Wv
    scores = Q @ K.T / np.sqrt(K.shape[-1])
    n = len(X)
    scores[np.triu(np.ones((n, n)), k=1).astype(bool)] = -np.inf
    return softmax(scores) @ V

class Cache:
    def __init__(self):
        self.keys, self.values = [], []

def attend_new_token(x_new, cache, Wq, Wk, Wv):  # this episode
    q, k, v = x_new @ Wq, x_new @ Wk, x_new @ Wv
    cache.keys.append(k)
    cache.values.append(v)
    K, V = np.stack(cache.keys), np.stack(cache.values)
    w = softmax(q @ K.T / np.sqrt(len(k)))
    return w @ V

rng = np.random.default_rng(0)
X = rng.normal(size=(5, 4))          # 5 tokens, 4 numbers each
Wq, Wk, Wv = (rng.normal(size=(4, 4)) for _ in range(3))

cache = Cache()
rows = [attend_new_token(x, cache, Wq, Wk, Wv) for x in X]
print(np.allclose(np.stack(rows), attention(X, Wq, Wk, Wv)))
print(len(cache.keys), np.stack(cache.keys).shape)
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. The naive loop repeats work · rewatch ▶ 0:08-0:21

1.1 9 new: 5 on step 1, then 1 on each of the 4 later steps. $35 - 26 = 9$. Step 1 processes the prompt for the first time; after that, only the one newly added token is new on each step.

1.2 46. The steps process 10, 11, 12 and 13 tokens: $10 + 11 + 12 + 13 = 46$.

1.3 False. Each step processes the whole text, which is one token longer than last time, so every step costs a little more than the one before.

1.4 B. The old tokens' work is recomputed on every step. The rest of this lesson is about not doing that.

2. Old tokens never change · rewatch ▶ 0:21-0:34

2.1 B. The new token looks back at everything (a new row); the old tokens never look ahead, so their rows are unchanged.

2.2 11 new cells, 6 with real weights. $6 \times 6 - 5 \times 5 = 11$. The new row (6 cells) is all real: the new token sees itself and the 5 before it. The 5 new cells above it are masked to zero.

2.3 False. *cat* never looks ahead, so nothing about it changes: its key and value are exactly the same as before.

2.4 No. The old tokens would now attend to the new one, so their outputs change, and later layers compute keys and values from those outputs. The cache only works because the past never sees the future.

3. The KV cache: one new row per step · rewatch ▶ 0:34-1:04

3.1 B. Everything about the old tokens is already in the cache; only the new token is computed.

3.2 Only the new token asks a question. A query is used to build that token's own row of the grid. Old rows are finished and never change, so their queries are not needed again.

3.3 (a) 5,050. (b) 100. Without a cache: $1 + 2 + \dots + 100 = 5,050$. With it, one token per step: 100. The longer the text, the bigger the saving.

3.4 False. Old keys and values never change, so the cached ones are exactly what a recomputation would give. Concept 7 checks this in code.

4. The price: memory · rewatch ▶ 1:04-1:20

4.1 18,432,000 (about 18.4 million). 18,432 numbers per token $\times$ 1,000 tokens.

4.2 (a) 262,144. (b) About 4.2 GB. $2 \times 32 \times 4,096 = 262,144$. Then $262,144 \times 8,000 \times 2$ bytes = 4,194,304,000 bytes $\approx$ 4.2 GB: gigabytes, as the video says.

4.3 B. Each token adds the same fixed amount (a key and a value per layer), so twice the tokens means twice the cache.

4.4 False. Every layer computes its own keys and values, so every layer has its own cache. That is the $\times 12$ in the formula.

5. Prefill and decode · rewatch ► 1:20-1:30

5.1 user sends a prompt → prefill → the cache holds the prompt's keys and values → decode. Prefill reads the prompt and fills the cache; only then can decode reuse it.

5.2 False. Prefill reads the whole prompt at once. One token at a time is decode.

5.3 The prompt is known in advance; generated tokens are not. Each new token must first be picked from the previous step's prediction before it can be processed.

5.4 (a) 1. (b) 3,686,400. Decode processes only the newest token. After prefill the cache holds $200 \text{ tokens} \times 18,432 \text{ numbers} = 3,686,400$.

6. Shrinking the cache: grouped-query attention · rewatch ► 1:30-1:41

6.1 4 per group; 4× smaller. $32 \div 8 = 4$ query heads per group, and 8 instead of 32 key/value heads are cached.

6.2 6,144. $2 \times 12 \text{ layers} \times (4 \text{ heads} \times 64) = 6,144$; three times smaller than 18,432, because $12 \div 4 = 3$.

6.3 False. The query heads stay the same; the keys and values are shared, and they are what the cache stores.

6.4 B. Sharing keys and values means fewer of them to store for every token.

7. The KV cache in code · rewatch ► 1:41-1:54

7.1 K: (7, 64). w: (7,). One cached key per token, stacked into rows; one weight for each cached token.

7.2 B. Future tokens have not been generated yet, so they are not in the cache.

7.3 (a) True · (b) 5 (5, 4) · (c) the first print becomes False (a) Feeding the tokens one at a time through the cache gives exactly the same five output rows as full causal attention over the whole grid. (b) One key per token: 5 keys of 4 numbers, stacked into a 5×4 matrix. (c) Without the mask, the full version lets each token see the tokens after it, but when the cache handles that token the later ones do not exist yet. The two agree only because the causal mask keeps the past from seeing the future (concept 2).